

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Case study

QUESTIONS:4

Total Marks:40

1.A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss**, **180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: Video Conferencing Quality Degradation

A multinational IT company's cloud-based video conferencing platform experiences voice distortion, frozen video, and delayed communication during peak hours. Monitoring shows 8% packet loss, 180 ms jitter, and fluctuating bandwidth.

1. Transport Protocol vs Application Layer — Root Cause Analysis

The symptoms described point primarily to the transport protocol layer, though application-level buffering strategy is a contributing amplifier. The evidence:

- **Packet loss (8%):** This is a network/transport-layer symptom of congestion, unreliable links, or an inappropriate reliability mechanism — not something the application layer creates directly.
- **Jitter (180 ms):** Variation in packet arrival time is caused by queuing delay and route/path variability in the network, and by how the transport protocol schedules and retransmits segments.
- **Fluctuating bandwidth:** Indicates congestion-control behavior at the transport layer reacting to a shared, variable-capacity link.

If the platform is using TCP for real-time media, TCP's built-in reliability (retransmission) and in-order delivery guarantee are themselves the root cause: TCP will pause delivery of all subsequent data until a lost segment is retransmitted and acknowledged (head-of-line blocking), which converts a small packet loss into a large, visible stall (frozen frames) and amplifies jitter far beyond the network's actual delay variation. Hence the problem is primarily a transport-protocol mismatch — using a reliable, ordered, congestion-controlled protocol (TCP) for delay-sensitive, loss-tolerant real-time media — compounded by application buffering that is not tuned for these conditions.

2. Justification Based on Real-Time Multimedia Requirements

Real-time audio/video has fundamentally different requirements from bulk data transfer:

Requirement	Real-Time Multimedia Need	TCP Behavior	UDP Behavior
Timeliness	Strict end-to-end delay budget (~150 ms one-	Retransmission + ACK wait adds unpredictable delay	No retransmission delay; packets delivered as soon as received

Requirement	Real-Time Multimedia Need	TCP Behavior	UDP Behavior
	way is the ITU-T G.114 target)		
Loss tolerance	Occasional lost frame/sample is acceptable and can be concealed	Cannot tolerate loss — always retransmits	Tolerates loss; app can conceal or interpolate
Ordering	Approximate ordering is enough (via timestamps in RTP)	Strict in-order delivery causes head-of-line blocking	No ordering guarantee — no blocking
Throughput vs latency	Prioritizes low, stable latency over guaranteed throughput	Congestion control (slow start, AIMD) can throttle and cause bursty delivery	No congestion backoff, so delivery timing stays predictable (but can worsen congestion if unmanaged)

Because real-time voice/video favors a fresh, timely packet over a perfectly complete one, the ITU-T and IETF standard architecture for conferencing (WebRTC, SIP-based systems) runs media over UDP using RTP/RTCP, while only signaling (call setup) uses TCP/TLS. Using TCP for the media stream itself directly contradicts the latency-over-reliability requirement of real-time multimedia, which explains the frozen frames (retransmission stalls) and distorted audio (jitter exceeding the client's playout buffer).

3. Recommended Protocol-Level and Application-Level Improvements

Protocol-level (Transport / Network)

- **Migrate media transport to UDP-based RTP/SRTP** (with RTCP for feedback) instead of TCP, so a lost or late packet is simply skipped rather than stalling the stream.

- **Use WebRTC with SReAM/GCC congestion control** which is designed specifically to adapt bitrate for real-time media without TCP-style throughput-maximizing backoff.
- **Enable Forward Error Correction (FEC) and packet-loss concealment** to reconstruct or approximate lost audio/video data without requesting retransmission.
- **Deploy Quality of Service (QoS) / DiffServ marking (EF class)** on the corporate network so voice/video packets are prioritized over bulk traffic during peak hours.
- **Use multiple TURN/STUN-reachable regional media relays (SFU architecture)** to shorten the path and reduce congestion-induced jitter and loss.

Application-level

- **Adaptive jitter buffer:** size the playout buffer dynamically based on measured jitter instead of a fixed buffer, trading a small amount of extra delay for smoother playback.
- **Adaptive bitrate / simulcast:** reduce video resolution or frame rate automatically when packet loss or bandwidth fluctuation is detected.
- **Redundant audio encoding (e.g., Opus in-band FEC):** duplicate critical audio data in a following packet so isolated losses don't cause audible gaps.
- **Bandwidth estimation and admission control:** throttle non-critical background data (screen-share resolution, file transfers) during active calls.

Case Study 2: Global DNS Resolution Redesign

A multinational software company's employees face long homepage load delays; DNS lookup averages ~800 ms. The company wants a DNS redesign that keeps availability high during server failures.

1. Reasons Behind High DNS Resolution Time

- **Centralized/distant authoritative servers:** if all DNS queries travel to one (or few) authoritative servers in a single region, users on other continents incur long round-trip propagation delay.
- **Deep resolution chain:** each uncached query may need to traverse root → TLD → authoritative name servers, multiplying round trips before an answer is returned.
- **Low or absent caching (low TTL or no local resolver cache):** if TTL values are set too low, recursive resolvers must repeatedly re-query authoritative servers instead of serving from cache.
- **No geographically distributed resolvers:** without Anycast or regional replicas, every query — regardless of the user's location — is routed to the same physical server, adding unnecessary network latency.
- **Recursive resolver overload:** a single/shared recursive resolver handling global query volume during peak hours can queue requests, adding processing delay on top of network delay.
- **No prefetching or local caching at the client/browser:** each new session repeats full resolution instead of reusing recently resolved records.

2. Proposed DNS Architecture (Target: < 50 ms Lookup)

A layered architecture combining Anycast DNS, DNS pre-fetching, and TTL optimization is proposed:

- **Anycast DNS deployment:** Publish the same authoritative IP address from multiple Points of Presence (PoPs) across continents (e.g., North America, Europe, Asia-Pacific). BGP routing automatically directs each user's query to the topologically nearest PoP, cutting propagation delay from hundreds of milliseconds to typically 10-30 ms.
- **Secondary/geo-redundant authoritative name servers:** Run at least two independent DNS provider networks (multi-provider Anycast, e.g., a primary and a secondary vendor) so a failure or attack on one constellation does not take resolution offline — queries automatically fail over to the healthy provider.

- **TTL optimization:** Set TTL for stable records (A/AAAA of core services) to 300-3600 s to maximize cache hits at recursive resolvers, while keeping TTL shorter (30-60 s) only for records that need fast failover (e.g., load-balancer VIPs), balancing cache efficiency against update agility.
- **DNS pre-fetching:** Use browser/OS-level DNS prefetching (dns-prefetch hints) and application pre-resolution for known-critical domains (auth server, API gateway, CDN) so the lookup happens in the background before it is actually needed on the critical path.
- **Local/regional caching resolvers (recursive resolver pools close to offices/data centers):** deploy resolver caches at each regional office or cloud region so most repeat queries are answered locally in under a few milliseconds.
- **Health-checked failover:** combine Anycast with active health checks so an unhealthy PoP is withdrawn from BGP announcements within seconds, redirecting traffic to the next-nearest healthy site.

3. Advantages and Limitations of the Proposed Architecture

Dimension	Advantages	Limitations
Performance	Anycast routes to nearest PoP, cutting lookup time typically below 50 ms; prefetching hides latency from the critical path entirely	Anycast performance depends on accurate BGP routing; suboptimal ISP peering can occasionally route to a farther PoP
Scalability	Query load is naturally distributed across many PoPs; adding a new region only requires announcing the prefix there	Requires coordination with multiple network operators/ASNs and careful capacity planning at each PoP
Availability	Multi-provider Anycast + health checks give resilience against single-provider outages, DDoS, or regional failures	Multi-provider setups increase operational and licensing cost and configuration complexity

Dimension	Advantages	Limitations
Consistency	TTL tuning balances freshness with cache efficiency	Longer TTLs mean DNS-level failover changes propagate more slowly to already-cached clients

Case Study 3: Distributed DNS for Global Cloud Traffic

A cloud provider's millions of global users see DNS queries taking ~800 ms because all requests hit a single primary DNS server, especially during promotional traffic spikes.

4. Impact of Centralized DNS Architecture on Application Performance

- **High latency for distant users:** every query, regardless of the requester's location, must traverse the full network path to one physical server, adding significant round-trip time for users far from that server.
- **Single point of failure:** if the primary server becomes unreachable or overloaded, all name resolution — and therefore all application access — fails simultaneously worldwide.
- **Poor scalability under load spikes:** during promotional events, sudden surges in concurrent users create queuing delay at the single server, worsening latency further and risking timeouts.
- **Cascading application-level delays:** since DNS resolution is typically the first step of any web/app request, an 800 ms lookup is added in full to page-load or API-call latency, directly affecting perceived responsiveness and can trigger client-side retries that add even more load.
- **Vulnerability to DDoS:** a centralized DNS server is an attractive, high-impact target — an attack against it can take down access to the entire global service.

5. Distributed DNS Solution — Design

- **Global Anycast network of authoritative DNS clusters:** deploy identical DNS server clusters at multiple PoPs across continents, all announcing the same IP address via Anycast/BGP so each user is automatically routed to the nearest, least-loaded cluster.
- **Hierarchical caching layer:** recursive resolvers positioned regionally (and CDN edge locations) cache answers close to users, so only cache-miss queries travel to the authoritative tier.

- **Multi-provider redundancy:** use two or more independent Anycast DNS providers concurrently (active-active) so total dependence on a single vendor's infrastructure is eliminated.
- **Load-based and latency-based traffic steering:** combine DNS-based global server load balancing (GSLB) with health checks so, beyond just DNS lookup speed, the returned application server IP is also the nearest/healthiest one, improving performance beyond just the DNS layer.
- **Elastic autoscaling of DNS clusters:** provision each regional cluster to autoscale ahead of known promotional events so query queuing does not recur under spikes.

6. Influence of TTL, Anycast Routing, and Caching on Performance and Fault Tolerance

Mechanism	Effect on Performance	Effect on Fault Tolerance
TTL values	Longer TTL = more cache hits at resolvers = fewer round trips to authoritative servers = lower average latency	Longer TTL delays propagation of failover changes; shorter TTL enables faster rerouting away from a failed server but increases query load
Anycast routing	Routes each query to the topologically nearest healthy cluster, minimizing propagation delay regardless of user location	If one PoP fails, BGP withdrawal automatically redirects traffic to the next-nearest PoP with no client-visible change in IP address
DNS caching	Answers served from local/recursive cache avoid the full resolution path, often resolving in under a few milliseconds	Reduces load on authoritative servers during spikes, insulating them from being overwhelmed, though stale cached data can briefly point to a failed endpoint until TTL expiry

Together, a moderate TTL (balancing freshness and cache efficiency), Anycast routing (proximity and automatic failover), and multi-tier caching (load absorption) directly address the case study's problems: they reduce the observed 800 ms centralized lookup time toward tens of milliseconds and remove the single point of failure represented by the primary DNS server.

Case Study 4: TCP Latency in a High-Frequency Trading Platform

An electronic stock trading platform sees order-acknowledgment latency rise from 1 ms to 40 ms during market open, along with increased TCP retransmissions, longer connection setup, and excessive buffering.

1. Three TCP-Related Factors Behind the Latency Increase

- **a) Flow control (receive window exhaustion):** At market open, order/acknowledgment volume spikes sharply. If the receiver's TCP receive buffer (and advertised window) is not sized for this burst, the sender is throttled by TCP flow control — it must pause and wait for window updates once the receiver's buffer fills, adding delay to every subsequent segment even though the network itself is not congested.
- **b) Nagle's algorithm:** Nagle's algorithm delays sending small TCP segments so they can be coalesced into larger packets, waiting either for enough data to fill a segment or for an ACK of previously sent small data. Trading messages (buy/sell orders, acknowledgments) are typically small and latency-critical; Nagle's algorithm can hold a small order message in the send buffer for up to the full ACK round-trip time, directly adding tens of milliseconds of unnecessary delay — a classic problem when Nagle's algorithm is combined with delayed ACKs on the receiving side.
- **c) SYN queue limitations (connection establishment backlog):** A burst of new client connections at market open can exceed the server's SYN backlog queue size. Once the queue fills, new SYN packets are silently dropped, forcing clients to retransmit their SYN after a timeout, which can add hundreds of milliseconds to connection establishment for late-arriving clients and increases the retransmission count network engineers observed.

2. How Each Factor Affects Transaction Processing

Factor	Effect in High-Frequency Trading
Flow control	Order and acknowledgment segments queue up waiting for receive-window space, so trade confirmations are delayed exactly when order volume — and the need for speed — is highest; can also trigger retransmission timers if the wait is long enough to look like loss.

Factor	Effect in High-Frequency Trading
Nagle's algorithm	Small, latency-sensitive order/ack messages are batched instead of sent immediately, so acknowledgment latency rises even when bandwidth and CPU are not the bottleneck — the delay is purely protocol-induced buffering.
SYN queue limitations	New trading sessions (e.g., additional client connections opened at market open) experience connection setup delay or failure and must retransmit SYNs, which both directly delays the affected client's first orders and adds background retransmission traffic that further stresses the network.

3. Recommended TCP Configuration Changes

- **Disable Nagle's algorithm (set TCP_NODELAY):** on all trading-session sockets so small order/acknowledgment messages are transmitted immediately instead of being buffered, eliminating the coalescing delay.
- **Disable or tune delayed ACKs (TCP_QUICKACK where supported):** so acknowledgments are not artificially deferred, which otherwise compounds with Nagle's algorithm at the peer even after TCP_NODELAY is set locally.
- **Increase TCP receive/send buffer sizes and enable window scaling:** size buffers to the expected peak burst volume at market open so flow control does not throttle bursts of order traffic; enable TCP window scaling (RFC 1323) to support large windows on high-bandwidth links.
- **Increase the SYN backlog queue and enable SYN cookies:** raise the listen backlog to accommodate the market-open connection burst, and enable SYN cookies so legitimate connection bursts are not dropped even if the queue is briefly full.
- **Tune retransmission timers (lower initial RTO where the network is a stable, low-latency LAN/co-location link):** so genuine losses are recovered faster without waiting on an overly conservative default retransmission timeout.

- **Consider UDP-based or kernel-bypass messaging for the most latency-critical path:** many real-world HFT systems move order-entry and market-data feeds off standard TCP/IP stack entirely (e.g., using UDP multicast for market data and kernel-bypass technologies such as DPDK/RDMA/Solarflare OpenOnload) once TCP tuning reaches its practical limits, since even a well-tuned TCP stack retains inherent OS-kernel and protocol overhead unsuitable for sub-millisecond targets.
- **Connection pooling / persistent sessions:** maintain long-lived TCP connections for trading sessions rather than repeatedly opening new ones, avoiding the SYN-queue and three-way-handshake cost altogether during peak order bursts.

Applying these changes converts the platform from a general-purpose TCP configuration (optimized for throughput and efficient use of small packets) to one optimized for the low-latency, small-message, bursty-connection profile that electronic trading actually requires — directly targeting the flow-control, Nagle, and SYN-queue causes identified above.